

BoardController

`BoardController` is the main runtime controller for an ArrowMagic board. It owns the board dimensions, procedural generation settings, lose conditions, optional mask-based spawning, optional arrow coloring behavior, and editor-mode support. It is also responsible for creating, restarting, and updating the active puzzle board.

Board

`width`

Type: `int`

Default: `8`

Sets the width of the board in cells.

If a spawn mask is assigned, the board dimensions may be overridden to match the mask texture size during restart.

`height`

Type: `int`

Default: `8`

Sets the height of the board in cells.

Like `width`, this may be overridden by the assigned spawn mask size when the board is rebuilt.

`seed`

Type: `int`

Default: `12345`

Seed used for procedural level generation.

Changing the seed changes the generated board layout. This is the primary way to generate alternate puzzle arrangements while keeping the same generation settings.

Generation

arrowCoverage

Type: float

Range: 0.05 - 1.0

Default: 0.95

Controls how much of the board should be filled with arrows during procedural generation.

Higher values generally produce denser boards with more arrows. Lower values leave more empty space.

twistiness

Type: float

Range: 0 - 1

Default: 0.50

Controls how twisty or directional the generated arrow paths feel.

Lower and higher values shift the character of the generated paths. This is one of the main controls for changing the style of a puzzle layout.

Lose Condition

blockedLoseLimit

Type: int

Minimum: 1

Default: 3

The number of unique blocked chains the player can accumulate before losing the level.

Once the blocked count reaches this limit, the board enters the lose state and the lose event is fired.

Arrow Colors (Optional)

ArrowColorMode

Type: `enum`

Controls how arrow colors are chosen.

Available modes:

- `UsePalette`
Arrows use the normal palette-based coloring.
- `UseSpawnMask`
Arrow colors are sampled from the spawn mask instead of using the palette alone.

`ArrowColor`

Type: `ArrowColorMode`

Read-only property

Returns the currently selected arrow color mode. This is the public accessor used by other systems.

`ArrowColorMask`

Type: `Texture2D`

Read-only property

Returns the texture used as the arrow color mask.

In this controller, the arrow color mask and the spawn mask are the same texture.

`ArrowColorMaskAlphaThreshold`

Type: `float`

Read-only property

Value: `0.5`

Alpha cutoff used when evaluating the color mask.

Pixels below this threshold are treated as not contributing to arrow color masking.

`ArrowColorMaskQuantizeSteps`

Type: `int`

Read-only property

Default: `16`

Controls color quantization for the arrow color mask before colors are applied.

- 1 = no quantization
- higher values such as 8, 16, or 32 produce cleaner, more palette-like color bands

This is useful when using painted or gradient masks and wanting more stylized results.

Runtime Board State

These are public runtime properties that are useful for UI, tools, and gameplay systems.

CanSpawnHere

Type: `bool[]`

Read-only property

A per-cell array indicating where arrows are allowed to spawn.

Its length is `width * height`. It is built from the assigned spawn mask when the board restarts. If no mask is assigned, all cells are allowed.

State

Type: `BoardState`

Read-only property

The current live board state.

This contains the generated or edited tiles and is the main board model used during play.

Rules

Type: `IRuleset`

Read-only property

The ruleset currently driving board interaction and solve logic.

Won

Type: `bool`

Read-only property

Whether the current board has been solved.

Lost

Type: `bool`

Read-only property

Whether the current board has entered the lose state.

MoveCount

Type: `int`

Read-only property

Returns the current number of moves stored in the undo stack.

Useful for HUD display.

CurrentSeed

Type: `int`

Read-only property

Returns the current active seed value. Useful for HUDs, debugging, and authoring tools.

BlockedUniqueCount

Type: `int`

Read-only property

The number of unique blocked chains recorded so far for the current board.

This is compared against `blockedLoseLimit` to determine when the player loses.

Public Events

These are useful for UI, animation, and gameplay listeners.

OnDeltaApplied

Fired after a move is successfully applied. Provides the `MoveDelta`.

OnWin

Fired when the puzzle is solved.

OnLose

Fired when the lose condition is reached.

OnRestart

Fired after the board is regenerated or restarted.

OnRedraw

Fired when the board should be visually rebuilt without a full gameplay restart.

OnEditorModeChanged

Fired when editor mode is toggled. Passes the new enabled state.

OnBlockedChanged

Fired whenever the unique blocked count changes.

Notes for documentation

A few fields in this script are technically part of the generation system but are currently hidden from the inspector, so they may or may not belong in your end-user docs depending on how public you want the docs to feel:

- `travelMode`
- `minPathLen`
- `maxPathLength`
- `validateWithGreedy`
- `spawnMask`
- `editedLevel`

For asset-store-facing docs, I'd probably split the writeup into:

1. **Inspector Variables**

2. **Runtime Read-Only Properties**
3. **Events**
4. **Editor-Only / Advanced Fields**

The next best step is for me to turn this into a more polished **user manual style version** with wording like “Use this when...” and “Recommended range...”, which is usually better for package documentation.

BoardView2D.cs

File

Let's now do the same for this script, BoardView2D.

Thought for 15s

Absolutely — here's the same style of documentation pass for **BoardView2D** based on the script you uploaded.

BoardView2D

BoardView2D is the main visual renderer for the ArrowMagic board. It builds the tile grid, positions tiles in world space, renders board colors, manages arrow overlays, coordinates intro animation visuals, and refreshes the board after moves or redraws. It also acts as the bridge between the **BoardController** game state and the visible 2D presentation.

Inspector References

controller

Type: **BoardController**

Reference to the active board controller.

This is the gameplay source for the board state, move events, restart events, editor mode state, win state, and color-mask settings. **BoardView2D** listens to this controller and redraws itself whenever the board changes.

tilePrefab

Type: **TileView2D**

Prefab used to create the visible board tiles.

Each cell on the board uses this prefab for rendering arrows, dots, heads, tails, and click handling. This prefab also supplies sprites used by the overlay systems, such as the arrow head sprite and straight tail sprite.

Layout

cellSize

Type: `float`

Default: `1.1`

Controls the spacing between board cells in world space.

This affects tile placement, path building, bump distances, and overlay rendering. Increasing it spreads the board out; decreasing it packs the board more tightly.

origin

Type: `Vector2`

Default: `Vector2.zero`

Base origin used by the board layout system.

During rebuild, the layout is centered from the current board dimensions, and this value is synchronized back from the layout so the inspector reflects the active board origin. In practice, this is more of a layout/debug value than a primary authoring control.

Arrow / Selection Animation

animationSpeed

Type: `float`

Default: `32`

Controls the playback speed of selection overlay animation.

This affects how quickly the animated arrow-selection visual moves along a chain when the player interacts with tiles. Higher values make the selection feel snappier; lower values make it feel slower and more deliberate.

Arrow Tail / Path Shape

cornerRadius

Type: `float`

Range: `0.0` - `0.49`

Default: `0.2`

Controls how rounded arrow overlay corners appear.

This value is passed into the board path builder and affects how smoothly paths bend around corners. Lower values create sharper turns; higher values create softer, more curved transitions.

Public Runtime Properties

These are the main public-facing values and helpers exposed by `BoardView2D`.

ArrowOverlayPool

Type: `ArrowOverlayPool2D`

Read-only property

Returns the shared overlay pool used by the board.

This pool supplies arrow overlay objects for board snakes, intro overlays, selection overlays, and editor preview strokes. Using one pool avoids repeatedly instantiating and destroying overlay objects at runtime.

IntroSamplesPerTile

Type: `int`

Read-only property

Returns the number of path samples used per tile for intro rendering.

Internally this is clamped to at least `2`. It represents how finely intro paths are sampled when building animated arrow reveal paths.

Public Methods

These are the methods most relevant to other systems, animation helpers, and editor tooling.

GetColorForIndex(int idx)

Returns the current display color for the tile at a given board index.

This is a convenience method that forwards to **BoardColorService2D**, falling back to white if no color service is present. Useful for systems that need the current visual tile color.

RedrawAll()

Forces the board to redraw all visible tiles from the current live board state.

This is used when the controller or editor changes the board without requiring a full restart. It also refreshes stable arrow colors and rebuilds snake overlays when needed.

GetHeadSprite()

Returns the arrow head sprite from the tile prefab.

Used by overlay systems that need to render arrow heads consistently with the main board tiles.

UnhideAllForIntro()

Makes all tiles visible again.

This is mainly used as a safety/helper method for intro workflows when tiles were temporarily hidden during reveal animation.

GetOverlayStraightSprite()

Returns the straight tail sprite from the tile prefab.

Used by arrow overlay systems so board snakes and selection overlays match the visual style of the tile prefab.

BuildIntroWorldPoints(List<Vector2Int> orderedCells, List<Vector3> outPts, BoardState s)

Builds the world-space path points used for intro animation.

This converts a chain of board cells into world positions that can be used to animate an intro arrow reveal.

SetAllDotsVisibleForIntro(bool visible)

Shows or hides dots on all tiles for the intro presentation.

This is used for the “full grid” intro look where dots may temporarily appear everywhere, even on arrows or blocked cells.

SetDotsVisibleForCells(List<Vector2Int> cells, bool visible)

Shows or hides dots only for a specific set of cells.

Used during intro reveal so dots can disappear only along the currently revealing chain.

HandlePointerTileClicked(int x, int y, Vector3 clickWorldPos)

Passes a pointer click into the selection system using both grid coordinates and world position.

This is useful for input systems that already know the clicked world point and want the selection animation to respond accurately.

EditorPreviewStroke(IReadOnlyList<Vector2Int> cells)

Displays a live editor preview of a drawn stroke.

When editing a level, this method shows a translucent overlay snake that follows the dragged cell path before the stroke is committed.

ClearEditorPreviewStroke()

Clears the current editor preview overlay.

Used when an editor drag ends, collapses to too few cells, or should otherwise stop showing the temporary preview path.

Important Internal-but-Documentable Behavior

These are not public inspector fields, but they are useful to explain in package docs because they define how `BoardView2D` behaves.

Tile pooling

`BoardView2D` reuses tile instances instead of destroying and recreating them every rebuild.

This improves runtime efficiency and avoids repeated allocation during board restarts or redraws.

Shared arrow overlay pool

All arrow overlays use one shared `ArrowOverlayPool2D`.

That includes board snakes, selection overlays, intro overlays, and editor preview strokes. This keeps overlay rendering centralized and more efficient.

Intro-aware snake rebuilding

If the intro animation is running, board snake rebuilds are queued until the intro finishes.

This prevents board snake overlays from fighting the intro visuals and ensures the final board snakes appear cleanly after the reveal.

Spawn-mask-aware dot suppression

When tiles are rebuilt, dots are suppressed on cells where `controller.CanShowDot(idx)` is false.

This makes the board view respect masked-out cells from the controller's spawn mask.

Arrow color mask support

If the controller is using `ArrowColorMode.UseSpawnMask`, the board view asks the color service to apply colors from the mask image.

This is where the controller's arrow color mask settings become visible in the 2D board rendering.

BoardInput2D

BoardInput2D handles player interaction with the 2D board. It reads pointer and press input from Unity's Input System, detects tile clicks, supports camera panning and zooming, and switches behavior when the board is in editor mode so drag input becomes a stroke for authoring arrow paths instead of gameplay input.

Inspector References

cam

Type: `Camera`

Reference to the camera used for converting screen input into world positions.

This camera is used for raycasting tiles, panning the board view, and zooming in or out. If no camera is assigned, the script falls back to `Camera.main` on enable.

controller

Type: `BoardController`

Reference to the active board controller.

This is used to check board state, editor mode, win/lose lockout, spawn-mask restrictions, and to submit editor actions such as tap-delete or stroke creation.

view

Type: `BoardView2D`

Reference to the active board view.

This is used for click forwarding, live editor preview strokes, and clearing temporary preview overlays. If not assigned, the script searches for a `BoardView2D` automatically on enable.

Input System

point

Type: `InputActionReference`

Input action used to read the current pointer position.

This should point to an action that returns a **Vector2** screen position, such as mouse position or touch position. It is used for raycasting, drag detection, panning, and editor stroke capture.

press

Type: **InputActionReference**

Input action used to detect pointer press start and release.

This action drives click/tap begin and end logic. It is also what starts and finishes editor strokes.

Raycast

tileLayerMask

Type: **LayerMask**

Default: **~0**

Controls which layers can be hit when raycasting for board tiles.

If your tiles are on a dedicated layer, setting this mask can help avoid accidental hits on unrelated colliders in the scene.

Panning

allowPanning

Type: **bool**

Default: **true**

Enables or disables drag-based camera panning.

When enabled, dragging beyond the configured threshold switches the interaction from a click into a pan gesture.

dragThresholdPixels

Type: `float`

Default: `10`

Minimum drag distance, in screen pixels, before a press is treated as a pan instead of a click.

Higher values make panning less sensitive and reduce accidental drags. Lower values make panning begin more quickly.

`panSensitivity`

Type: `float`

Default: `1`

Controls how strongly the camera moves while panning.

Higher values make the camera move farther for the same pointer drag distance. Lower values produce a gentler pan.

Zoom

`allowZoom`

Type: `bool`

Default: `true`

Enables or disables camera zoom input.

When enabled, the script supports both desktop scroll zoom and mobile pinch zoom.

`minOrthographicSize`

Type: `float`

Default: `3`

Minimum zoom level for orthographic cameras.

Smaller orthographic sizes zoom in closer, so this sets the closest allowed orthographic zoom.

`maxOrthographicSize`

Type: float

Default: 40

Maximum zoom level for orthographic cameras.

Larger orthographic sizes zoom farther out, so this sets the farthest allowed orthographic zoom.

minPerspectiveFov

Type: float

Default: 20

Minimum field of view for perspective cameras.

Lower FOV values appear more zoomed in. This sets the closest allowed perspective zoom.

maxPerspectiveFov

Type: float

Default: 70

Maximum field of view for perspective cameras.

Higher FOV values appear more zoomed out. This sets the farthest allowed perspective zoom.

wheelZoomSpeed

Type: float

Default: 0.5

Controls how quickly mouse-wheel or trackpad-scroll zoom changes the camera.

Positive scroll input is interpreted as zooming in through an internal inverted delta. Increase this value for stronger zoom steps per wheel movement.

pinchZoomSpeed

Type: float

Default: 0.01

Controls how quickly pinch gestures zoom the camera on touch devices.

Increase this value to make pinch zoom feel stronger; reduce it for more precise zoom control.

Public-Facing Behavior

While `BoardInput2D` does not expose many public properties, it defines several important behaviors that are worth documenting for users of the package.

Gameplay click behavior

When the player presses and releases without panning, the script treats the interaction as a tile click and forwards it to `BoardView2D`, which then routes the interaction into gameplay selection logic.

Input lock when won or lost

If the board is not in editor mode and the controller reports `Won` or `Lost`, normal interaction is blocked.

Any active click, pan, or editor interaction is canceled immediately when this lockout is detected. This is how post-win and post-lose boards stop accepting gameplay input.

Camera framing suppression

Whenever the player begins interacting manually—clicking, panning, or zooming—this script tells `BoardCameraFramer2D` to stop auto-framing until the next level.

This prevents the framing system from fighting manual camera control.

Editor stroke capture

In editor mode, pressing on a valid tile starts a stroke instead of standard gameplay input.

As the pointer moves across adjacent cells, the script builds a list of stroke cells and asks `BoardView2D` to show a live preview overlay. On release, the stroke is either:

- applied as a quick single-tile delete, or
- committed as a full stroke to the controller

Only valid spawnable cells may be included, and the stroke is constrained to 4-way adjacent movement.

Touch pinch zoom

On touch devices, two active touches are interpreted as a pinch zoom gesture.

Starting a pinch immediately cancels any current click, pan, or editor stroke so the gesture cleanly takes over.

Scroll zoom

On desktop and laptop devices, mouse wheel input is used for zoom. On many trackpads, two-finger scrolling also feeds into this same path.

Helper Methods Worth Mentioning in Docs

These are not inspector variables, but they are useful to describe in technical documentation.

CancelCurrentInteraction()

Clears the current press, pan, editor stroke, and preview state.

Used whenever input must be forcefully stopped, such as after losing, winning, or when a pinch gesture takes over.

HandleScrollZoom()

Processes wheel or trackpad scroll zoom input when zoom is enabled.

HandleTouchPinchZoom()

Processes two-finger pinch zoom on touch devices.

ApplyZoomDelta(float delta)

Applies zoom to either orthographic size or perspective field of view, depending on the current camera type.

TryGetTileAtScreenPos(Vector2 screenPos)

Raycasts from screen position to find the tile under the pointer.

ScreenToWorldOnBoard(Vector2 screenPos)

Converts a screen position into a world point on the board plane using the assigned camera.

AddStrokeCellIfNew(Vector2Int cell)

Adds a new editor stroke cell only if it is valid, new, mask-allowed, and 4-way adjacent to the previous cell.

BoardColorService2D

`BoardColorService2D` is the board's centralized color service. It owns the active arrow palette for the board, computes stable colors for connected arrow chains, supports permanent per-tile color overrides, optionally applies arrow colors from a mask texture, and stores hit-tint settings used by board feedback systems.

This script is the main source of truth for tile color lookup during board rendering.

Purpose

Use `BoardColorService2D` when you want the board to support:

- palette-driven arrow coloring
- stable chain-based colors
- mask-based arrow coloring
- permanent per-tile color overrides
- configurable hit-tint settings

It is designed to keep all board color behavior in one place so other systems, like `BoardView2D`, can simply ask for the final display color of a tile.

Inspector Variables

Arrow Palette

`currentPalette`

Type: `Palette`

The active palette used for arrow coloring on this board.

When stable arrow colors are rebuilt, connected arrow chains are assigned colors from this palette. If no palette is assigned, the service falls back to white.

Blocked / Hit Tint

`tintOnHit`

Type: `bool`

Default: `true`

Enables or disables hit-tint behavior.

This setting does not animate anything by itself. It simply exposes whether other board-view or feedback systems should apply tinting when arrows are hit.

`hitTint`

Type: `Color`

Default: `Color.white`

Tint color used when hit-tint feedback is enabled.

Other systems can read this value and temporarily tint a tile or arrow when it is struck or blocked.

Public Read-Only Properties

`TintOnHit`

Type: `bool`

Read-only property

Returns whether hit tinting is currently enabled.

`HitTint`

Type: `Color`

Read-only property

Returns the currently configured hit tint color.

CurrentPalette

Type: `Palette`

Read-only property

Returns the currently assigned palette used for stable arrow coloring.

PermanentTileColors

Type: `Dictionary<int, Color>`

Read-only property

Returns the dictionary of permanent per-tile color overrides.

These overrides always take priority over stable chain colors when the final tile color is requested.

Public Events

PaletteChanged

Type: `event Action<Palette>`

Raised whenever the active palette is changed through `SetPalette(...)`.

This allows other systems to refresh visuals when the board palette changes.

Public Methods

Init(BoardController controller)

Initializes the color service with the active board controller.

This stores the controller reference and immediately rebuilds stable arrow colors for the current board state. This should be called before the service is used for runtime color lookup.

SetPalette(Palette palette, bool rebuild = true)

Sets the active palette for the board.

Behavior:

- does nothing if the requested palette is already active
- stores the new palette
- optionally rebuilds stable arrow colors
- raises `PaLETTEChanged`

This is the main runtime method for changing the board's palette.

`ClearAllPermanentOverrides()`

Removes all permanent per-tile color overrides.

After this is called, tiles return to their stable chain colors or white fallback.

`ClearPermanentOverride(int idx)`

Removes the permanent override for one tile index.

`SetPermanentOverride(int idx, Color c)`

Assigns a permanent color override to one tile index.

This color takes priority over the tile's stable chain color until cleared.

`TryGetStableColor(int idx, out Color c)`

Attempts to get the stable chain color for a tile.

Returns `true` if the tile currently has a stable chain color; otherwise returns `false`. This does not check permanent overrides.

`GetTileColor(int idx)`

Returns the final display color for a tile.

Priority order is:

1. permanent tile override
2. stable chain color
3. white fallback

This is the main color lookup method other systems should use when asking what color a tile should currently display.

ApplyArrowColorMaskFromImage(Texture2D mask, float alphaThreshold, int quantizeSteps)

Applies colors from an image mask to arrow tiles.

Behavior:

- only arrow tiles are affected
- samples color from the matching board position in the texture
- ignores pixels whose alpha is at or below `alphaThreshold`
- optionally quantizes RGB values
- stores the result as permanent tile overrides

This is the main method used for image-based arrow coloring.

RebuildStableArrowColors()

Recomputes stable chain colors for the current board.

This scans the board, collects connected arrow chains, computes a stable hash key for each chain, and assigns a palette color to that chain. Every tile in the same chain receives the same stable color.

ApplyHitTintSettings(bool enableTint, Color tint)

Updates the hit-tint settings at runtime.

This provides a simple runtime API for changing whether hit tint is used and what tint color should be applied.

Important Internal Behavior

These are not inspector fields, but they are important for understanding how the service works.

`_stableArrowColors`

Stores the current stable chain color assignment per tile index.

This is rebuilt from the board state whenever stable colors are recalculated.

`_chainColorByKey`

Stores a mapping from computed chain hash key to assigned color.

This ensures that all tiles in the same chain get the same color during a rebuild.

`_nextPaletteIndex`

Tracks which palette color should be assigned next as new chains are processed.

This causes connected chains to cycle through the palette colors in order.

`_permanentTileColors`

Stores permanent tile color overrides.

These overrides win over stable arrow colors whenever `GetTileColor(...)` is called.

Behavior Notes

Single active palette

This version of the service owns a single `currentPalette` directly.

Unlike the earlier palette-manager-based setup, the service no longer stores or cycles through an array of palettes itself. Palette switching happens by assigning a new `Palette` object.

Stable chain coloring

Arrow colors are assigned per connected arrow chain, not per individual tile.

The service collects a full connected chain using `ArrowChainUtility.CollectFullChain(...)`, computes a stable hash from the tile indices, and assigns one palette color to that full chain.

Palette fallback is white

If `currentPalette` is missing or empty, stable chain colors fall back to `Color.white`.

Permanent overrides always win

If a tile has a permanent override color, that color is returned even if the tile also has a stable chain color.

Mask coloring only affects arrows

`ApplyArrowColorMaskFromImage(...)` ignores all non-arrow tiles. Empty cells and other tile types are not assigned override colors by the mask.

Quantization range

`quantizeSteps` is clamped to the range 1 to 64.

- 1 behaves like unquantized passthrough
- higher values create more stepped, posterized color bands

This is useful for producing cleaner color regions from gradient-based masks.

Stable-color rebuild resets chain assignment state

Whenever `RebuildStableArrowColors()` runs, it clears prior chain-color bookkeeping and starts palette assignment from the beginning again.

Palette changes can optionally skip rebuild

`SetPalette(...)` includes a `rebuild` parameter.

That means a system can change the palette reference immediately and choose whether to rebuild stable colors right away or defer that work.

Internal Helper Methods Worth Mentioning

`ResolveArrowPalette()`

Returns the active palette as a `Color[]` if available.

If no valid palette exists, it returns `null`.

`ComputeChainKey(HashSet<int> chain)`

Builds a stable hash key from the sorted tile indices of a connected chain.

This key is used to assign one stable color per chain during rebuild.

Quantize01(float v, int steps)

Quantizes a normalized channel value into stepped increments.

Used internally by mask-based arrow coloring.

BoardCameraFramer2D

BoardCameraFramer2D is an optional camera helper for **BoardView2D**. Its job is to automatically position the camera so the full board fits on screen, with configurable padding. It can snap instantly or animate into place, and it can temporarily disable itself once the player starts manually interacting with the board.

Purpose

Attach this component when you want the board camera to auto-frame the puzzle at level start or after a rebuild.

Important behavior:

- it only works with **perspective cameras**
 - orthographic cameras are ignored
 - it can be suppressed once the player starts clicking or panning, so the framing system does not fight manual camera control
-

Inspector Variables

cameraPaddingInCells

Type: float

Default: 2.0

Padding around the board when computing the camera frame, measured in board cells.

This adds extra space around the puzzle so the board does not sit flush against the screen edges. The value is converted into world units by multiplying it by the board layout's `CellSize`. Larger values frame the board more loosely; smaller values frame it more tightly.

`animateCamera`

Type: `bool`

Default: `true`

Enables animated camera movement when framing is requested with `animate = true`.

If disabled, the camera snaps directly to the computed target position even when animated framing is requested.

`cameraMoveDuration`

Type: `float`

Default: `0.6`

Duration, in seconds, for animated framing moves.

This controls how long the camera takes to move from its current position to the framed target position. Smaller values feel snappier; larger values feel smoother and more cinematic.

`cameraMoveEase`

Type: `AnimationCurve`

Default: `EaseInOut(0,0,1,1)`

Easing curve used for animated camera movement.

This controls how the camera interpolates over time during an animated move. The default ease-in-out starts gently, speeds up, then slows near the end.

Implementation Options

`useCameraFitPerspective`

Type: `bool`

Default: `false`

Chooses which framing calculation method to use.

When enabled, the component uses `CameraFitPerspective.FrameBoard(...)` instead of its own internal distance-to-fit calculation. This is useful if your project has a preferred shared perspective-fitting helper and you want this component to rely on that implementation.

Public Methods

These are the main public-facing methods exposed by the component.

`SetCamera(Camera cam)`

Assigns a specific camera to use for framing.

If no override camera is assigned, the component falls back to `Camera.main`. Use this when your board should be framed by a non-main camera.

`SuppressUntilNextLevel()`

Disables further framing until re-enabled.

This is meant to be called as soon as the player starts interacting manually, such as clicking, dragging, or panning. It prevents the framer from moving the camera again during the current level, and it also stops any in-progress camera move started by this component.

`AllowFramingForLevel()`

Re-enables framing for the current level.

Call this before framing a newly started or restarted level so the component is allowed to move the camera again.

`StopMoveIfAny()`

Stops any currently running camera movement coroutine.

This is useful if something else needs to take over camera motion immediately.

`FrameToBoard(BoardLayout2D layout, BoardState s, bool animate)`

Frames the camera so the current board fits on screen.

This is the main runtime method. It:

- reads the board world bounds from the layout
- adds padding in world units
- computes the target camera position
- either snaps or animates to that position

Important conditions:

- does nothing if framing is currently suppressed
 - does nothing if no layout exists
 - does nothing if there is no camera
 - does nothing for orthographic cameras
-

Behavior Notes

These are useful to mention in end-user documentation because they explain how the framer behaves in practice.

Perspective only

This component only frames **perspective** cameras.

If the camera is orthographic, `FrameToBoard` exits immediately and no framing is applied. So this component is intended for perspective presentation only.

Manual interaction disables framing

Once `SuppressUntilNextLevel()` is called, the framer will no longer move the camera until `AllowFramingForLevel()` is called.

This is important for usability because it prevents auto-framing from overriding the player's own camera adjustments during play.

Snap vs animate

Framing only animates when all of these are true:

- `animate` was passed as `true`
- `animateCamera` is enabled

- `cameraMoveDuration` is greater than a tiny minimum
- the component is active and enabled

Otherwise, the camera snaps directly to the target position.

Padding is based on cell size

Padding is authored in **cells**, not world units.

That makes the framing behavior scale naturally with different board layouts, since the actual world padding is computed from `cameraPaddingInCells * layout.CellSize`.

Internal fit calculation assumptions

The internal framing math assumes the camera is axis-aligned and looking down the **-Z axis**.

If your camera setup is heavily customized or tilted, you may prefer enabling `useCameraFitPerspective` and routing through a dedicated fit helper.

Internal Helper Methods Worth Mentioning

These are not inspector variables, but they explain how the component works.

`AnimateCameraPosition(...)`

Coroutine that interpolates the camera position over time using the configured duration and ease curve.

`ComputePerspectiveDistanceToFitRect(...)`

Calculates the required camera distance needed to fit the board rectangle plus padding, based on camera field of view and aspect ratio.

BoardAuthoringTools

BoardAuthoringTools is a lightweight helper component used for board authoring workflows. Its main purpose is to hold and auto-resolve a reference to the local **BoardController** so editor tooling or companion authoring scripts can reliably access the active board controller on the same GameObject.

Purpose

Attach **BoardAuthoringTools** to the same GameObject as **BoardController** when you want a simple authoring-side access point to the controller.

This script does not generate content, modify level data, or render anything by itself. It mainly acts as a convenience wrapper around a **BoardController** reference.

Inspector Variables

controller

Type: **BoardController**

Serialized reference to the board controller used by authoring tools.

If this field is not assigned, the script automatically tries to find a **BoardController** on the same GameObject when accessed or when reset/auto-assigned.

Public Properties

Controller

Type: **BoardController**

Read-only property

Returns the active **BoardController** reference.

Behavior:

- if **controller** is already assigned, it returns that reference

- if `controller` is missing, it calls `GetComponent<BoardController>()`
- the found result is cached back into the serialized field

This makes the property the preferred way for other scripts to access the controller through this helper.

Public Methods

`AutoAssign()`

Attempts to auto-assign the `controller` field from the same `GameObject`.

This is a simple convenience method for editor or setup workflows. If the field is empty, it fills it using `GetComponent<BoardController>()`.

Unity Lifecycle Behavior

`Reset()`

Unity reset/setup helper that auto-assigns the `controller` reference.

When the component is first added or reset in the Inspector, it tries to populate the `controller` field from the same `GameObject`. This makes setup faster and reduces the chance of a missing reference.

Behavior Notes

Same-GameObject dependency

This script assumes the `BoardController` lives on the same `GameObject`.

Its lookup path is always `GetComponent<BoardController>()`, so it does not search children, parents, or the wider scene.

Convenience wrapper only

`BoardAuthoringTools` is intentionally minimal.

It does not contain level editing logic itself. Instead, it provides a stable place for authoring-related systems to get the board controller reference.

Safe inspector setup

Because the field is serialized and also auto-resolved, the script supports both:

- manual assignment in the Inspector
- automatic local assignment during setup

That makes it useful as a low-friction editor utility component.

BoardDots2D

`BoardDots2D` is an optional visual helper component for the board's dot layer. It owns the animated dot pulse settings, ripple settings, and win-ring effect settings. It does not create tiles itself; instead, it works with the tiles already created by `BoardView2D` and updates their dot color, visibility, and scale over time.

If this component is missing from the `BoardView2D` GameObject, dot animation effects are simply absent.

Purpose

Use `BoardDots2D` when you want the board's dots to feel more alive and reactive.

This component is responsible for:

- ambient pulsing of tile dots
 - ripple feedback from a chosen center cell
 - intro-time dot visibility control
 - a board-wide win-ring dot animation before the final level-won signal is raised
-

Inspector Variables

Pulse

dotA

Type: `Color`

Default: `RGBA(1, 1, 1, 0.15)`

First color used by the ambient dot pulse.

The pulsing animation blends between `dotA` and `dotB` over time. Lower alpha values make the dots feel softer and more subtle.

dotB

Type: `Color`

Default: `RGBA(1, 1, 1, 0.35)`

Second color used by the ambient dot pulse.

This is typically a brighter or more visible version of `dotA`, giving the dots a breathing or glowing motion.

dotPulseSpeed

Type: `float`

Default: `1.2`

Speed of the ambient pulse animation.

Higher values make the dots pulse faster. Lower values produce a slower, calmer effect.

Ripple

rippleSpeed

Type: `float`

Default: `6`

Speed at which the ripple band expands outward from the triggered center cell.

Higher values make the ripple travel faster across the board.

rippleWidth

Type: `float`

Default: `1.25`

Width of the ripple band, measured in board-cell distance.

Larger values create a broader, softer ripple. Smaller values produce a tighter, more defined wavefront.

rippleStrength

Type: `float`

Default: `0.75`

How strongly the ripple boosts the current pulse value.

Higher values make the ripple more visible against the normal ambient pulse. Lower values make the ripple more subtle.

Win Ring

WinRingSettings

Type: `struct`

A grouped settings block controlling the board's win-ring animation.

enabled

Type: `bool`

Default: `true`

Enables or disables the win-ring effect.

If disabled, the component skips the ring animation and goes straight to raising the level-won callback once conditions are met.

duration

Type: float

Default: 0.65

Duration of the win-ring animation in seconds.

ringWidthCells

Type: float

Default: 1.25

Thickness of the animated ring, measured in board-cell distance.

Larger values create a wider and softer ring. Smaller values create a sharper band.

baseColor

Type: Color

Default: RGBA(1, 1, 1, 0.15)

Base dot color used outside the active ring.

ringColor

Type: Color

Default: RGBA(1, 0.8365737, 0, 0.75)

Highlight color used at the strongest part of the ring.

This is the color dots blend toward as the ring passes over them.

maxScale

Type: float

Default: 2.5

Maximum dot scale used at the strongest part of the ring.

Higher values make the ring feel punchier and more celebratory.

Public Methods

These are the main public-facing methods used by the board view and other systems.

Configure(BoardController controller, Func<TileView2D[]> getTiles, Action redrawAll, Action raiseLevelWon)

Configures the component with the board controller, tile accessor, redraw callback, and win callback.

This is the main setup method called by **BoardView2D** once the board is ready. It stores the required references and immediately builds its internal tile cache.

RebuildCache()

Rebuilds the cached list of tile indices that currently have valid **TileView2D** instances.

This should be called whenever the tile array or board state changes significantly. It also recomputes the maximum board distance used by ripple timing.

StopMoveIfAny()

Stops any currently running win-ring coroutine.

Used when the component is disabled or when another system needs to interrupt the active win-dot animation.

TriggerRipple(Vector2Int centerCell)

Starts a ripple effect from the given board cell.

The ripple then expands outward over time during normal **Update()** processing.

MarkWinPending()

Marks the win VFX as pending, but does not fire it immediately.

This is used when the board has entered a win state but the system wants to wait until overlays or other visual elements have finished first.

TryFirePendingWinVfx(int activeOverlaysCount)

Attempts to fire the pending win effect if conditions are satisfied.

The win effect only fires when:

- a win was marked pending
- it has not already fired
- the controller exists and the board is actually won
- there are no active overlays remaining

If the win ring is enabled, it starts the ring coroutine. It also invokes the supplied level-won callback.

SetAllDotsVisibleForIntro(bool visible)

Forces intro-dot visibility on all existing tiles.

This is used for intro choreography where the normal tile logic is temporarily overridden.

SetDotsVisibleForCells(List<Vector2Int> cells, bool visible)

Forces intro-dot visibility only for a specific set of cells.

Useful when revealing or hiding dots along a path or a staged intro sequence.

Runtime Behavior

Update()

During runtime, **BoardDots2D** updates dot animation every frame if it has been configured correctly.

If the controller or tile accessor is missing, it safely does nothing. If valid tiles are present, it ensures its cache is up to date and then animates the dots.

OnDisable()

Stops any active win-ring coroutine when the component is disabled.

Important Behavior Notes

Optional component

This script is fully optional.

If it is not present, the board still works normally — you just do not get pulse, ripple, or win-ring dot effects.

Works through existing `TileView2D` instances

`BoardDots2D` does not render dots directly.

Instead, it updates each tile through methods such as:

- `SetEmptyDotColor`
- `SetDotVisible`
- `SetDotColor`
- `SetDotScale01`
- `ResetDotScale`
- `SetIntroDotVisible`

Suppressed dots are respected

If a tile reports `DotSuppressed`, this component skips its normal animation logic for that tile.

During the win ring, suppressed dots are explicitly hidden and have their scale reset.

Ripple is layered on top of pulse

The ripple does not replace the ambient pulse. It adds to it.

Internally, the component computes the normal pulse value first, then boosts that value inside the ripple band. That keeps the effect feeling integrated rather than like a completely separate system.

Ripple auto-expires

A ripple is automatically cleared once it has traveled beyond the farthest relevant board distance plus a small buffer.

So the effect does not need manual cleanup after being triggered.

Win effect can wait for overlays

The final win VFX is intentionally deferred until overlay activity has ended.

That prevents the win ring from competing visually with active arrow overlays or reveal animations.

Win ring animates from board center

The ring expands from the geometric center of the board outward toward the corners.

Dots near the active ring band are tinted toward `ringColor` and scaled up toward `maxScale`, then reset at the end.

Final redraw after win ring

When the win-ring coroutine completes, all dot scales are reset and the component asks the board to redraw.

This ensures the board returns cleanly to its normal post-effect appearance.

Internal Methods Worth Mentioning

These are not inspector variables, but they explain how the component works.

`EnsureCache()`

Checks whether the tile cache is still valid and rebuilds it if the board state or tile count changed.

`AnimateDots()`

Applies the current pulse and optional ripple to all cached dot tiles by updating their colors.

`WinDotRing()`

Coroutine that animates the expanding win ring across the board, updating dot color and scale over time.

GameEvents

`GameEvents` is a static event container that provides a shared set of gameplay and UI signals. It lets different systems communicate without needing direct references to one another. For example, board logic can raise an event when the player wins, and UI, audio, VFX, or analytics systems can respond by subscribing to that event.

Because it is a **static** class, it does not need to be added to a `GameObject` in the scene. Other scripts simply subscribe to its events or call its raise methods.

Purpose

Use `GameEvents` as a central messaging layer for common gameplay signals.

This is especially useful for:

- UI reactions
- sound effects
- particle/VFX triggers
- animation listeners
- loose coupling between gameplay and presentation systems

Instead of calling event delegates directly throughout the codebase, this script provides dedicated `Raise...()` methods so event dispatch stays consistent and easy to maintain.

Core Outcome Events

`LevelWon`

Type: `Action`

Raised when the level is completed successfully.

`LevelLost`

Type: `Action`

Raised when the player reaches the lose condition.

Arrow Interaction Events

`ArrowBlocked`

Type: `Action<Vector3>`

Raised when an arrow tries to move but is blocked by an obstacle.

The `Vector3` payload is the world position where the blocked interaction occurred. This is useful for spawning hit effects, sound effects, or screen feedback at the correct location.

ArrowExit

Type: `Action<Vector3>`

Raised when an arrow finishes exiting the board.

The `Vector3` payload is the world position of the exit event. This is useful for exit effects, trails, sounds, or score feedback.

Input / Flow Events

ArrowSelected

Type: `Action<Vector3>`

Raised when the player taps or clicks an arrow.

The `Vector3` payload is the world position of the selection. This can be used by VFX, selection indicators, or audio feedback systems.

UndoClicked

Type: `Action`

Raised when the Undo button is clicked.

ButtonClicked

Type: `Action`

Raised for a general button click.

This is useful as a shared UI click signal for simple sound or animation feedback when multiple buttons should trigger the same response.

Submit

Type: `Action`

Raised when a submit action occurs.

StartClicked

Type: `Action`

Raised when the Start button is clicked.

NextClicked

Type: `Action`

Raised when the Next button is clicked.

Public Raise Methods

These methods are the intended way to dispatch events from other systems.

RaiseLevelWon()

Raises the `LevelWon` event.

RaiseLevelLost()

Raises the `LevelLost` event.

RaiseArrowBlocked(Vector3 worldPos)

Raises the `ArrowBlocked` event with a world position payload.

RaiseArrowExit(Vector3 worldPos)

Raises the `ArrowExit` event with a world position payload.

RaiseArrowSelected(Vector3 worldPos)

Raises the `ArrowSelected` event with a world position payload.

`RaiseUndoClicked()`

Raises the `UndoClicked` event.

`RaiseButtonClicked()`

Raises the `ButtonClicked` event.

`RaiseSubmit()`

Raises the `Submit` event.

`RaiseStartClicked()`

Raises the `StartClicked` event.

`RaiseNextClicked()`

Raises the `NextClicked` event.

Behavior Notes

Static utility class

`GameEvents` is declared as a `static` class, so it exists globally and is not instantiated. Systems can subscribe from anywhere in code.

Loose coupling

This event hub helps separate systems that produce events from systems that react to them. For example, the board controller can raise a win event without knowing which UI, SFX, or VFX systems are listening.

World-position payloads for feedback

Several events provide a `Vector3 worldPos` argument so effects can be spawned at the right location in the scene. This is especially useful for arrow selection, blocked hits, and exits.

Preferred raise pattern

This script explicitly recommends using the provided `Raise...()` methods instead of calling `?Invoke()` from scattered locations. That keeps event dispatch centralized and easier to refactor later.

Subscription cleanup

Because these are static events, listeners should unsubscribe properly, usually in `OnDisable` or `OnDestroy`, to avoid stale subscriptions or unexpected callbacks. This is a best-practice note for your docs rather than something enforced by the script itself.

TileView2D

`TileView2D` is the visual component attached to each individual board tile. It stores that tile's grid coordinates, exposes tile sprites for other board systems, forwards click events, and manages the visibility, color, and scale of the tile's empty-dot renderer. At the moment, its rendering logic is intentionally lightweight and focuses mainly on dot presentation.

Inspector Variables

`emptyDot`

Type: `SpriteRenderer`

`SpriteRenderer` used for the tile's background / empty-dot display.

This is the main renderer that `TileView2D` directly controls. The script uses it to:

- show or hide the dot
 - tint the dot color
 - scale the dot for simple pulse or emphasis effects
-

Public Sprite References

These are public sprite references exposed on the tile prefab for use by other systems.

head

Type: `Sprite`

Sprite used as the arrow head visual reference.

`BoardView2D` and overlay systems can query this sprite from the tile prefab so overlay arrows match the same art style as the board tiles.

tail

Type: `Sprite`

Sprite used as the arrow tail / straight segment visual reference.

Like `head`, this is mainly a shared art reference for overlay and snake-rendering systems.

Public Read-Only Properties

Cell

Type: `Vector2Int`
Read-only property

Returns this tile's grid coordinates as a `Vector2Int`.

X

Type: `int`
Read-only property

Returns the tile's X coordinate on the board grid.

Y

Type: `int`
Read-only property

Returns the tile's Y coordinate on the board grid.

DotSuppressed

Type: `bool`

Read-only property

Returns whether the tile's dot is currently being suppressed.

When suppression is active, the dot is forced off regardless of normal render logic or intro overrides.

Public Events

Clicked

Type: `event Action<int, int>`

Raised when this tile is clicked through `Click()`.

The event sends the tile's grid coordinates (`x`, `y`). If nothing is subscribed, the tile falls back to calling `BoardController.ClickCell(x, y)` directly.

Public Methods

`Init(BoardController controller, int x, int y)`

Initializes the tile with its controller reference and board coordinates.

This is usually called by `BoardView2D` when building or reusing tile instances.

`Click()`

Triggers the tile click behavior.

Behavior:

- if something is subscribed to `Clicked`, the event is raised
- otherwise, the tile directly calls `controller.ClickCell(_x, _y)`

This makes the tile usable either through event-driven click handling or direct controller forwarding.

SetHidden(bool hidden)

Temporarily hides or unhides this tile's visuals.

This is mainly used while an overlay snake is animating, so the “real” arrow can disappear while the animated overlay plays on top. The method only updates the hidden state; the actual visible result is resolved later by `Render()`.

SetEmptyDotColor(Color c)

Sets the color of the empty-dot renderer.

SetDotVisible(bool visible)

Back-compatible helper that shows or hides the dot.

Internally this just calls `SetDotActive(visible)`.

SetDotColor(Color c)

Back-compatible helper for setting dot color.

Internally this forwards to `SetEmptyDotColor(c)`.

SetDotSuppressed(bool suppressed)

Enables or disables dot suppression.

When suppression is turned on, the dot is immediately forced off. This is useful for temporary effects that should prevent dots from showing even if normal render logic would want them visible.

SetIntroDotVisible(bool visible)

Enables intro-dot override mode and sets the forced dot visibility.

When intro override is active, normal “empty tile” visibility rules are ignored and the dot uses this override state instead, while still respecting suppression.

ClearIntroDotOverride()

Disables intro-dot override mode.

After this is called, dot visibility goes back to normal render logic.

SetDotScale01(float t01, float maxScale = 1.6f)

Scales the dot between its base size and a larger size.

- `t01` is normalized from 0 to 1
- `maxScale` sets the largest allowed scale multiplier

This is useful for pulse, emphasis, or selection effects on the dot.

ResetDotScale()

Restores the dot to its original cached scale.

Render(TileState t, Color arrowColor)

Updates the tile's visible state from the current tile data.

In the current implementation, this method is focused only on dot visibility. It does **not** directly render arrow sprites here.

Current dot logic:

- if dot suppression is active, the dot is forced off
- otherwise, if intro override is active, intro override decides visibility
- otherwise, the dot is visible only when the tile is `Empty` and not hidden

The `arrowColor` parameter is currently unused in this implementation, but it keeps the method compatible with the broader board rendering flow.

Important Behavior Notes

Current rendering scope is intentionally narrow

Right now, `TileView2D` mainly controls the empty-dot background.

The script even notes that if arrow or pipe visuals are reintroduced here later, the dot logic should remain isolated. That means this class is currently more of a lightweight per-cell dot/view helper than a full arrow renderer.

Hidden state affects normal dot rendering

When `_hidden` is true, a normally empty tile will not show its dot during standard rendering.

This is mainly for overlay animation workflows, where the real tile should visually disappear while a temporary animated version is shown.

Suppression overrides everything

Dot suppression has the highest priority.

If suppression is active, the dot stays off even if:

- the tile is empty
- intro override says it should be visible
- another system tries to show it directly

Intro override bypasses normal empty-tile rules

When intro override is active, the tile no longer decides dot visibility based on tile type.

Instead, it uses the explicitly assigned intro visibility value, which allows intro systems to choreograph exactly which dots appear and disappear during reveal sequences.

Cached SetActive optimization

The script caches the dot's last active state and avoids redundant `SetActive` calls.

This is a small optimization that helps reduce unnecessary GameObject activation churn during frequent redraws or animation updates.

Cached base scale

The dot's original local scale is cached the first time scale effects are used.

That allows the script to apply temporary scale changes and reliably restore the original size later.

LevelIntroAnimator2D

`LevelIntroAnimator2D` is an optional board-intro playback component. Its job is to take the current board state, convert it into revealable intro entries, sequence those entries according to the configured intro settings, and play the animated reveal using overlay arrows. When the sequence completes or is stopped, it restores the normal board view and fires a completion event.

If this component is present on the same `GameObject` as `BoardView2D`, the board view uses it during rebuild instead of showing board snakes immediately.

Purpose

Use `LevelIntroAnimator2D` when you want a level to reveal itself with an animated intro instead of appearing instantly.

This component is responsible for:

- preparing the board for intro playback
 - building intro entries from the current board state
 - ordering those entries according to intro settings
 - revealing each entry through `ArrowIntroRevealer`
 - restoring the board after playback
 - notifying listeners when the intro is finished
-

Inspector References

`boardView`

Type: `BoardView2D`

Reference to the board view used for intro playback.

This is used to prepare and restore board visuals during the intro. The animator calls methods such as:

- `UnhideAllForIntro()`
- `SetAllDotsVisibleForIntro(true)`
- `RedrawAll()`
- `SetDotsVisibleForCells(...)`

If this reference is missing, `Play()` attempts to auto-resolve it from the same `GameObject` using `GetComponent<BoardView2D>()`.

controller

Type: `BoardController`

Reference to the active board controller.

This is used to access the current `BoardState`. If the controller is missing or its state is null, `Play()` exits without starting the intro.

Settings

settings

Type: `IntroAnimationSettings`

Main intro configuration asset for the reveal sequence.

This settings object controls behavior such as:

- start ordering mode
- random seed for ordering
- stagger timing
- ease curve used during reveal

If `settings` is missing, `Play()` logs a warning and does not run the intro.

Public Events

OnIntroFinished

Type: `event Action`

Raised when the intro sequence has fully finished its cleanup and the board has been restored.

This event fires in the `finally` block of the intro coroutine, so it is also part of the cleanup path after playback work completes. It is used by `BoardView2D` to know when it is safe to rebuild board snake overlays after the intro has finished.

Public Methods

`Play()`

Starts the intro animation for the current board.

Behavior:

- auto-resolves `boardView` if needed
- checks that `controller` and `controller.State` are valid
- checks that `settings` is assigned
- stops any currently running intro
- starts the main intro coroutine using the current board state

This is the main entry point used to begin intro playback.

`Stop()`

Stops the currently running intro and all active reveal coroutines.

It also performs immediate board cleanup by:

- clearing the main routine reference
- stopping all tracked reveal coroutines
- unhiding all board tiles
- redrawing the board
- cleaning up intro overlays through the builder

Use this when you need to cancel the intro cleanly before it finishes on its own.

`SetSettings(IntroAnimationSettings s)`

Replaces the active intro settings object.

Behavior:

- if an intro is currently running, it stops that intro first
- stores the new settings reference

- clears the cached `IntroEntryBuilder` so it will be rebuilt with the new settings later

This is the runtime-safe way to swap intro settings.

Main Runtime Flow

`Run(BoardState s)`

This private coroutine is the heart of the intro system.

Its flow is:

1. Prepare the board for intro playback
 - unhide everything
 - force dots visible for intro
 - redraw the board
2. Build intro entries
 - uses `IntroEntryBuilder`
3. Order the entries
 - uses `IntroOrderer.Apply(...)`
4. Start reveal coroutines
 - one reveal per entry using `ArrowIntroRevealer.RevealOne(...)`
5. Optionally stagger reveals
 - based on `settings.stagger`
6. Wait long enough for the longest reveal plus total stagger time
7. Cleanup in `finally`
 - restore dots for revealed cells
 - clear routine tracking
 - unhide and redraw the board
 - clean up overlays
 - invoke `OnIntroFinished`

This structure makes the intro resilient because cleanup happens even if something exits the routine early.

Important Internal State

These are not inspector variables, but they are useful to explain because they define how the component behaves.

`_revealRoutines`

Tracks all per-entry reveal coroutines currently running.

This allows `Stop()` to cancel every active reveal, not just the main sequence coroutine.

`_routine`

Stores the currently running main intro coroutine.

If non-null, an intro is in progress.

`_builder`

Cached `IntroEntryBuilder` instance used to build intro entries and clean up its overlays afterward.

It is lazily created the first time the intro runs, and reset to null when `SetSettings(...)` is called so a new builder will be created for the new settings.

Behavior Notes

Optional component

This component is optional.

If it is not present, `BoardView2D` skips intro playback and shows board snake overlays immediately instead. If it is present, `BoardView2D` clears board snakes, starts the intro, and waits for `OnIntroFinished` before rebuilding snake overlays.

Auto-resolve for board view only

`Play()` auto-finds `BoardView2D` if `boardView` is missing, but it does not auto-find the controller.

So in practice:

- `boardView` can be left empty if the component is on the same `GameObject`
- `controller` still needs to be correctly assigned beforehand

Safe cleanup path

The intro uses a `try/finally` structure, which means board restoration and overlay cleanup are part of the guaranteed shutdown path of the coroutine.

That is important because intro visuals temporarily override board visibility and dot presentation.

Stagger is optional

Staggering only happens when `settings.stagger > 0`.

If stagger is zero or below, all entry reveals are started immediately and the sequence simply waits for the longest reveal duration.

Longest-reveal wait logic

The coroutine computes the maximum reveal duration across all entries, then adds total stagger time when stagger is enabled.

That ensures the intro sequence waits long enough for the final reveal to finish before cleanup begins.

Dots are restored per entry

In cleanup, the animator calls `boardView.SetDotsVisibleForCells(entries[i].cells, true)` for every entry before restoring the full board.

This helps return intro-hidden cells to their intended visible state before the full redraw.

Relationship to Other Systems

BoardView2D

`BoardView2D` detects this component during rebuild.

If found, it:

- marks intro as running
- clears current board snake overlays
- calls `Play()`
- queues board snake rebuilding for after `OnIntroFinished`

When the event fires, `BoardView2D` rebuilds the board snake overlays.

IntroEntryBuilder

Used to turn the board state into a list of `IntroEntry` objects and to clean up any overlays it owns afterward.

IntroOrderer

Used to reorder the built entries according to the configured intro start order and random seed.

ArrowIntroRevealer

Used to animate each intro entry's overlay arrow along its point path using the configured ease and duration.

ModeAudioListener

`ModeAudioListener` is an audio event listener that connects global gameplay events to sound effects. It subscribes to the events defined in `GameEvents` and plays the appropriate audio clips from an assigned `SfxLibrary`.

This component acts as the bridge between **gameplay events** and **audio playback**, allowing sound effects to be triggered automatically when key actions occur such as arrow interactions, UI button presses, and win/lose states.

Purpose

Attach `ModeAudioListener` to a `GameObject` in your scene when you want sound effects to respond automatically to gameplay and UI events.

Typical responsibilities include:

- playing win/lose sounds
- playing arrow interaction sounds
- playing UI button sounds
- centralizing event-driven audio playback

Because it listens to `GameEvents`, this component does not need direct references to board systems or UI systems. It simply reacts to global events.

Inspector Variables

`sfx`

Type: `SfxLibrary`

Reference to the sound effects library used by this listener.

The `SfxLibrary` defines the audio clips, volumes, and pitch ranges for each supported gameplay event. Each entry can optionally randomize pitch and select from multiple clips for variation.

If no library is assigned, the listener safely ignores all events.

Unity Lifecycle

`OnEnable()`

Subscribes the listener to all relevant `GameEvents`.

The component begins listening for gameplay and UI events when it becomes active. Events include win/lose states, arrow interactions, and UI button actions.

`OnDisable()`

Unsubscribes the listener from all `GameEvents`.

This ensures the component stops receiving callbacks when it is disabled, preventing stale event references or unexpected audio triggers.

Gameplay Event Handlers

These methods respond to gameplay events and play the corresponding sound effect.

OnWin()

Plays the **win** sound effect.

Triggered when the level is successfully completed.

OnLose()

Plays the **lose** sound effect.

Triggered when the player reaches the lose condition.

OnArrowBlocked(Vector3 pos)

Plays the **arrow blocked** sound effect at the world position where the blocked interaction occurred.

This allows spatialized sound to originate from the location of the blocked arrow.

OnArrowExit(Vector3 pos)

Plays the **arrow exit** sound effect at the arrow's exit position.

Used for feedback when arrows successfully leave the board.

OnArrowSelected(Vector3 pos)

Plays the **arrow selected** sound effect at the tile position where the arrow was selected.

UI Event Handlers

These methods respond to UI interactions.

OnUndo()

Plays the **undo** sound effect.

Triggered when the Undo button is clicked.

OnClick()

Plays the general **button click** sound effect.

This is used for generic UI interaction feedback.

OnSubmit()

Plays the **submit** sound effect.

Triggered when a submit action occurs.

OnStart()

Plays the **start** sound effect.

Triggered when the Start button is clicked.

OnNext()

Plays the **next** sound effect.

Triggered when the Next Level button is clicked.

Internal Playback Method

Play(SfxLibrary.SfxEntry entry, Vector3 worldPos)

This helper method performs the actual audio playback.

Process:

1. Verifies the `SfxLibrary` reference exists.
2. Samples a clip, volume, and pitch from the requested `SfxEntry`.
3. Uses `SfxUtil.PlayOneShot3D` to play the clip in 3D space.

This design allows each sound entry to support:

- random clip selection
 - pitch variation
 - volume variation
 - positional audio
-

Behavior Notes

Event-Driven Audio

`ModeAudioListener` relies entirely on `GameEvents` to trigger sound playback. This means audio behavior can be modified without changing gameplay systems.

Safe Failure Behavior

If the `SfxLibrary` is not assigned or an entry fails to provide a valid clip, the event simply produces no sound instead of generating an error.

Spatialized Audio

Some sounds are played at specific world positions (such as arrow interactions). This allows sound effects to feel anchored to the board rather than coming from the camera.

Centralized Audio Routing

Using a single listener for gameplay audio helps keep sound logic centralized and avoids duplicating audio calls across gameplay scripts.

ModeVfxListener

ModeVfxListener is a visual effects event listener that connects gameplay events to particle or visual effects. It listens to the global **GameEvents** system and spawns the appropriate VFX from an assigned **VfxLibrary**.

This component is the visual-effects counterpart to **ModeAudioListener**. While the audio listener triggers sound effects, this script triggers particle effects and visual feedback for the same gameplay events.

Purpose

Attach **ModeVfxListener** to a GameObject in your scene when you want visual feedback to respond automatically to gameplay events.

Typical uses include:

- win/lose celebration effects
- arrow interaction effects
- impact or selection effects
- testing and previewing VFX during development

Because it listens to **GameEvents**, the component stays loosely coupled to gameplay logic.

Inspector Variables

VFX Library

vfx

Type: **VfxLibrary**

Reference to the library of visual effects used by this listener.

The library contains particle system prefabs (or similar visual effects) for events such as:

- win
- lose
- arrow blocked
- arrow exit
- arrow selected

Each entry in the library can optionally randomize or sample from multiple VFX prefabs.

If this reference is missing, the listener simply ignores events.

Spawn Space Settings

These settings control how and where visual effects appear.

vfxCameraRelative

Type: `bool`

Default: `true`

Determines how outcome VFX (win and lose) are positioned.

If **true**

- the effect spawns relative to the camera
- the prefab's authored local offset is preserved
- typically used for screen-space or UI-like effects

If **false**

- the effect spawns in world space at the listener's transform position

Arrow interaction VFX always spawn in world space.

debugCooldownSeconds

Type: `float`

Default: `0.15`

Prevents rapid repeated debug triggers.

When debug VFX testing is enabled, this cooldown ensures effects cannot be triggered faster than the specified interval.

Set to **0** to disable the cooldown.

Debug VFX Testing

These options allow developers to trigger VFX manually during development.

`enableDebugVfxTest`

Type: `bool`

Enables keyboard testing for win/lose VFX.

When enabled (only in editor or development builds), the configured keys will trigger the associated effects.

`testWinKey`

Type: `KeyCode`

Default: `UpArrow`

Keyboard key used to trigger the **win** VFX during debugging.

`testLoseKey`

Type: `KeyCode`

Default: `DownArrow`

Keyboard key used to trigger the **lose** VFX during debugging.

Runtime Behavior

Camera Resolution

Awake()

Caches a reference to the main camera if one has not already been assigned.

The camera reference is used when spawning camera-relative effects.

Event Subscriptions

OnEnable()

Subscribes to the following gameplay events:

- **LevelWon**
 - **LevelLost**
 - **ArrowBlocked**
 - **ArrowExit**
 - **ArrowSelected**
-

OnDisable()

Unsubscribes from all gameplay events to prevent stale callbacks when the component is disabled.

Event Handlers

These methods respond to gameplay events and spawn the corresponding VFX.

OnWin()

Triggers the **win** visual effect.

Behavior depends on `vfxCameraRelative`:

- **true** → camera-relative effect
 - **false** → world-space effect at the listener's position
-

OnLose()

Triggers the **lose** visual effect.

Uses the same camera-relative or world-space behavior as the win effect.

OnArrowBlocked(Vector3 pos)

Spawns the **arrow blocked** effect at the world position where the arrow was blocked.

OnArrowExit(Vector3 pos)

Spawns the **arrow exit** effect at the position where the arrow left the board.

OnArrowSelected(Vector3 pos)

Spawns the **arrow selected** effect at the tile location where the arrow was clicked.

Debug Testing Behavior

Update()

When running in the **Editor or Development builds**, and if `enableDebugVfxTest` is enabled:

- pressing the **win test key** triggers the win effect
- pressing the **lose test key** triggers the lose effect

The `debugCooldownSeconds` setting prevents rapid repeated triggering.

This system is designed to allow artists or designers to preview outcome VFX without needing to play through a full level.

Internal Playback Methods

`PlayWorld(VfxLibrary.VfxEntry entry, Vector3 worldPos)`

Spawns a visual effect at a specific world-space position.

Used for:

- arrow interactions
 - world-space win/lose effects when camera-relative mode is disabled
-

`PlayCameraRelative(VfxLibrary.VfxEntry entry)`

Spawns a visual effect relative to the camera using the prefab's authored local offset.

Used primarily for win/lose effects that should appear in screen space.

Behavior Notes

Event-driven VFX

`ModeVfxListener` listens to the same event system used by the audio listener. This keeps gameplay systems independent of visual presentation.

Safe fallback behavior

If the `VfxLibrary` reference is missing or a VFX entry cannot provide a prefab, the event simply produces no effect instead of causing errors.

Camera-relative effects

Outcome VFX often work best as camera-relative effects (for example, screen flashes or confetti bursts). The `vfxCameraRelative` option allows easy switching between screen-space and world-space presentation.

Debug testing tools

The debug testing system is restricted to **Editor and Development builds**, preventing accidental triggering in release builds.
